Day 3 morning — Monte Carlo and Markov chains

Bayesian statistics for PhD researchers · Lecturer: Lu Cheng

Thursday 12 November 2026 · Pontoppidanstræde 101, room 1.001

Averages of draws · Monte Carlo error · Markov chains · the Metropolis algorithm · effective sample size

Today

08:45	**Lecture A**	Monte Carlo error · Markov chains · the Metropolis algorithm
09:45	**Exercise A**	Monte Carlo error in R · run and read a Metropolis sampler · question set
11:10	**Peer review A**	Partner interviews · plenary · everyone back at 11:10
11:55	**Lunch**	
12:30	**Lecture B**	Stan: model blocks, HMC and NUTS, chains, diagnostics
13:30	**Exercise B**	Run Stan, check against the exact answer, read diagnostics · question set
15:00	**Peer review B**	Partner interviews · plenary · everyone back at 15:00
15:45	**Daily diary**	Mentimeter: 3 things learned, 3 not yet familiar

Follow-up from yesterday

From the Day 2 diaries and plenary

Fill in on Wednesday evening: the two or three points that came up most often under “not yet familiar”.

1. …
2. …
3. …

By the end of this lecture you can

1. **Estimate** a posterior mean or probability as an average of draws
2. **Quantify** the Monte Carlo error of such an estimate (MCSE)
3. **Explain** what a stationary distribution is, and why MCMC uses it
4. **Run** a Metropolis sampler and read its trace plot
5. **Judge** a chain by its effective sample size, not its length

Back to the bus: we know the answer

Day 2

14 of 20 departures on time, prior Beta(2, 2)

→ posterior Beta(16, 8)

Why simulate, then?

From Day 4 on, models have several parameters and no formula for the posterior. Then draws are all we have. Today we learn the tools on a case where we can check every answer.

exact summary	value
posterior mean	0.667
posterior SD	0.094
$P(\theta > 0.6 \mid y)$	0.763
90% interval	0.504 – 0.814

```
In R: 16/24,  qbeta(c(0.05, 0.95), 16, 8),
1 - pbeta(0.6, 16, 8)
```

A known posterior is a test case: it tells us whether the computer got it right.

Integrals become averages

$$E[\, h(\theta) \mid y \,] \approx (1/S) \cdot \Sigma\, h(\theta^{(s)}), \quad \theta^{(1)}, \ldots, \theta^{(s)} \text{ drawn from } p(\theta \mid y)$$

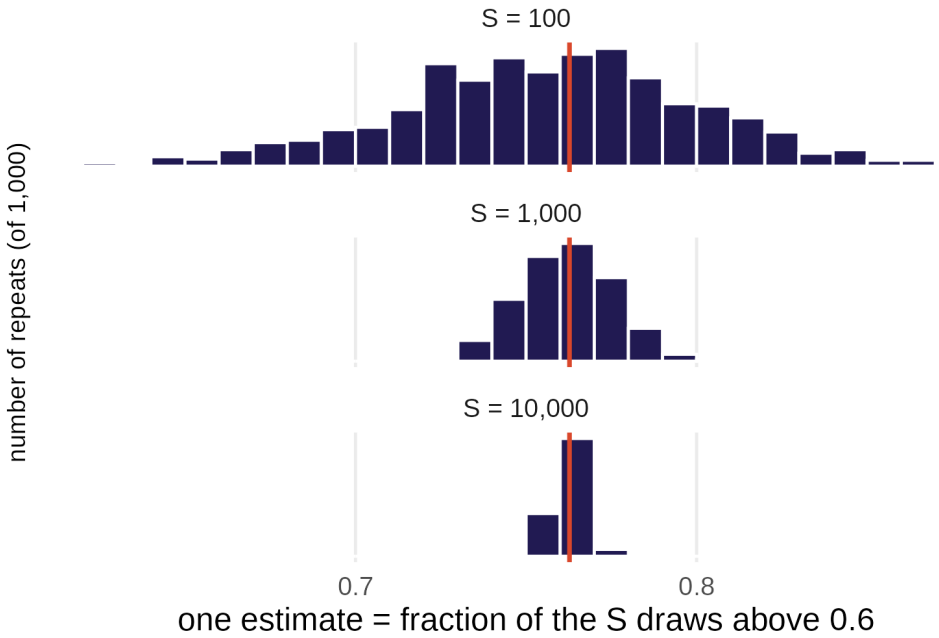
```
draws <- rbeta(4000, 16, 8)    # S = 4000 draws

# posterior mean: h(θ) = θ
mean(draws)
# a probability:  h(θ) = 1{θ > 0.6}
mean(draws > 0.6)
# a 90% interval
quantile(draws, c(0.05, 0.95))
```

- A probability is the mean of a TRUE/FALSE vector.
- Any summary, any function of θ: the same recipe.
- You used this on Day 2 for prediction.

Choose the quantity first (mean, probability, interval), then compute it from the draws.

The estimate itself is random




```
# one repeat = one estimate
draws <- rbeta(S, 16, 8)
mean(draws > 0.6)
```

S	SD of the 1,000 estimates	formula
100	0.044	0.043
1,000	0.014	0.013
10,000	0.0041	0.0043

Monte Carlo standard error = SD of the estimate

MCSE = SD(h(θ)) / √S

for a probability p: √(p(1 − p) / S)

The draws always follow Beta(16, 8). What shrinks is the spread of the estimate: 100× more draws → 10× smaller error.

Where does √S come from?

The estimate is an average of S independent values: $\hat{h} = (h_1 + h_2 + \ldots + h_S) / S$, with $h_s = h(\theta^{(s)})$

1. Each h_s has the same variance $\sigma^2 = \mathrm{Var}(h(\theta))$.
2. Independent values: variances add. $\mathrm{Var}(h_1 + \ldots + h_S) = S \cdot \sigma^2$
3. Divide by S. Rule: $\mathrm{Var}(cX) = c^2 \cdot \mathrm{Var}(X)$, here $c = 1/S$
   $\mathrm{Var}(\hat{h}) = (1/S^2) \cdot S\,\sigma^2 = \sigma^2 / S$
4. Take the square root: $\mathrm{MCSE} = \mathrm{SD}(\hat{h}) = \sigma / \sqrt{S}$

```
h <- draws > 0.6
sd(h) / sqrt(length(h))   # MCSE in R
```

Simple case: a probability

$h_s = 1$ if $\theta^{(s)} > 0.6$, else 0

→ Bernoulli(p), so $\sigma^2 = p(1 - p)$

MCSE = $\sqrt{(p(1 - p) / S)}$

p = 0.763, S = 1,000:

$\sqrt{(0.763 \times 0.237 / 1000)} = 0.013$

(simulated SD of the estimates: 0.014 ✓)

Step 2 needs independent draws. For a Markov chain, replace S by the effective sample size (later today).

Two kinds of uncertainty — do not mix them up

Posterior SD = 0.094

Our uncertainty about θ given the data.

Shrinks with more DATA (more departures).

It is the answer to the scientific question.

MCSE of the mean ≈ 0.0015 (S = 4000)

How far our computed number may be from the exact one.

Shrinks with more DRAWS.

It is a property of the computation only.

- Report the posterior SD or interval as the result.
- Infinitely many draws give the exact posterior — not the true θ.

When we cannot draw directly

What we can always compute

likelihood × prior = $p(y \mid \theta) \cdot p(\theta)$

for any value of θ — even with many parameters.

What is hard

the normalising constant $p(y) = \int p(y \mid \theta)\, p(\theta)\, d\theta$,

and therefore direct draws like rbeta().

Way out

Build a random process that we can run step by step, using only likelihood × prior, and whose long-run behaviour is the posterior.

The tool for this is a Markov chain. First: what is a Markov chain?

A Markov chain: tomorrow depends on today

Weather on an island (invented): each day is sunny or rainy.

today ↓ tomorrow →	sunny	rainy
sunny	0.9	0.1
rainy	0.2	0.8

Transition probabilities: each row sums to 1.

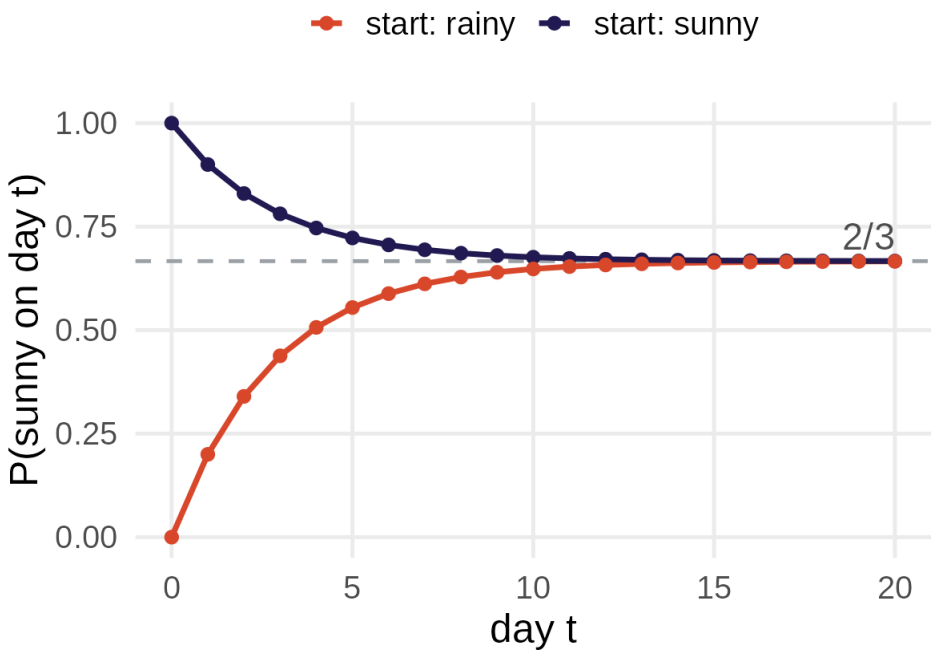
Markov property

To draw tomorrow we need only today.

The days before today do not matter.

```
P <- matrix(c(0.9, 0.1,
              0.2, 0.8), nrow = 2, byrow = TRUE)
today <- 1             # 1 = sunny, 2 = rainy
tomorrow <- sample(1:2, 1, prob = P[today, ])
```

The stationary distribution



- Start on a sunny day or on a rainy day: after about 15 days P(sunny) = 2/3 either way.
- 2/3 sunny, 1/3 rainy is the stationary distribution π.

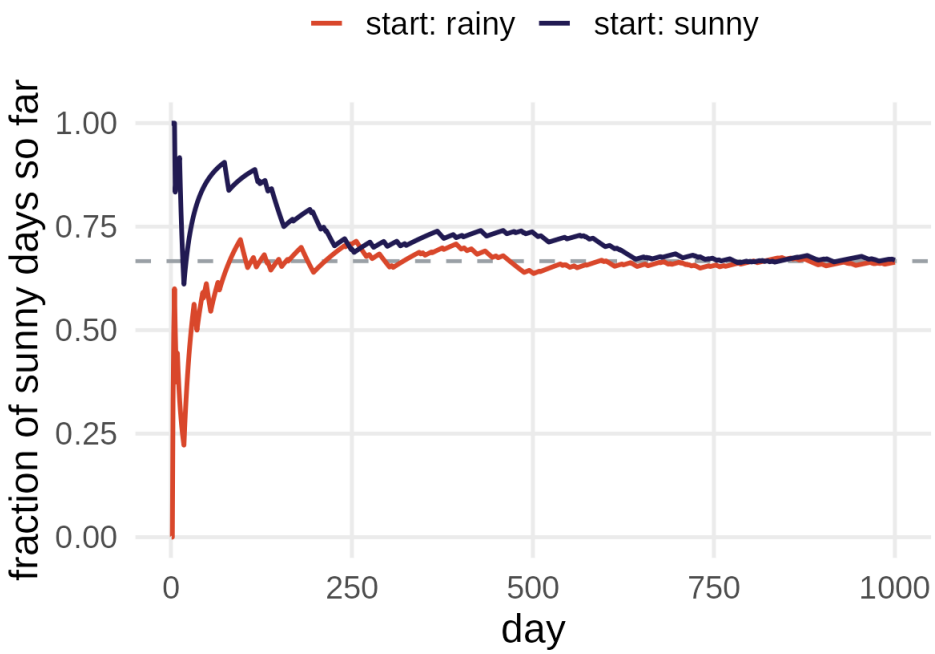
Stationary: it does not change

If today is sunny with prob. 2/3, so is tomorrow:

$2/3 \times 0.9 + 1/3 \times 0.2 = 2/3$ ✓

A Markov chain has a long-run distribution that does not depend on the start.

One long chain settles at π



- Simulate one chain for 1,000 days and count the sunny days so far.
- The fraction settles at 2/3, from either start.
- So averages over one long chain estimate averages under π.
  - Neighbouring days are similar: more days are needed than with independent draws.

This is what we need: a chain run long enough behaves like draws from π.

MCMC: turn the question around

Weather: rule → π

Given the transition rule, which distribution does the chain settle into?

Answer: 2/3 sunny.

MCMC: π → rule

Given the target π = posterior p(θ | y), design a rule whose stationary distribution is that posterior.

- Run the chain long; drop the first part, before it has forgotten its start (warmup).
- Use the visited θ values like posterior draws: means, probabilities, intervals.
- Metropolis (next) is the simplest such rule; Stan's HMC (this afternoon) is another.

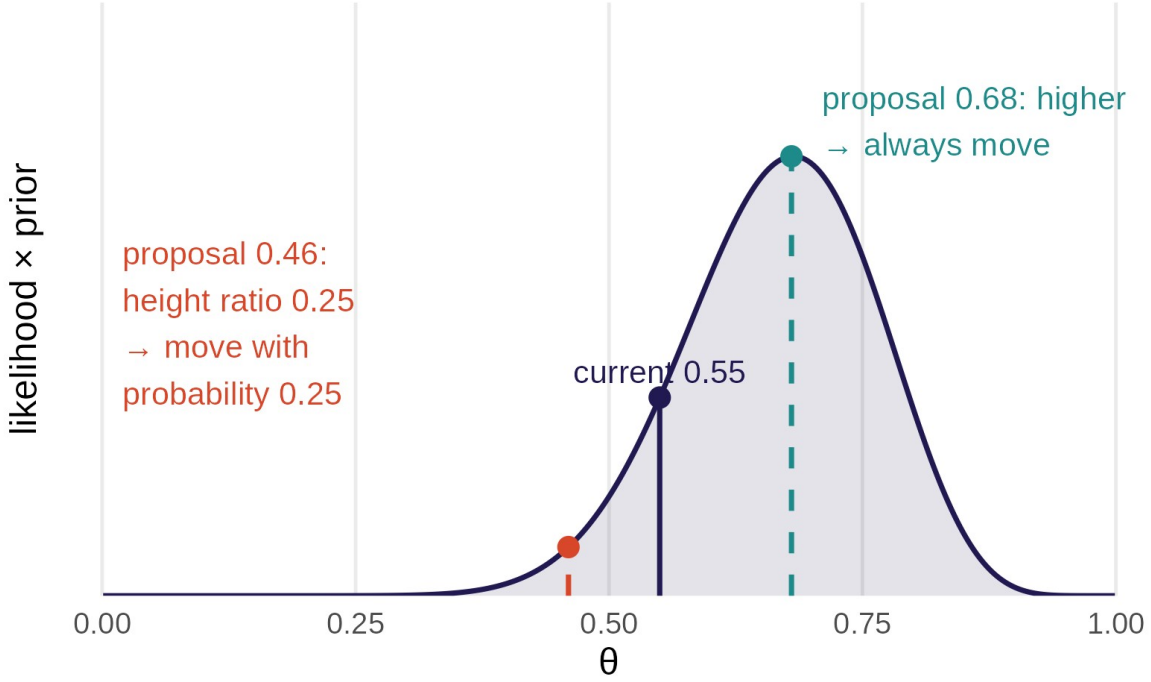
MCMC = Markov chain Monte Carlo: Monte Carlo averages over the steps of a Markov chain.

The Metropolis algorithm

1. Start somewhere: $\theta = 0.5$
2. Propose a nearby value: $\theta^* = \theta + \text{noise}$, $\text{noise} \sim \text{Normal}(0, \text{step})$
3. Compare heights: $r = [\text{likelihood} \times \text{prior at } \theta^*] / [\ldots \text{ at } \theta]$
4. Move to θ^* with probability $\min(1, r)$; otherwise stay at θ
5. Record the current value (also when it stayed!) and go to 2

In r the unknown constant p(y) appears above and below the line, so it cancels.

One step, drawn



- Uphill: always move.
- Downhill to a quarter of the height: move one time in four.
- So low regions are visited, but rarely.
  - Proposals outside (0, 1) have height 0 → always rejected.

The Metropolis sampler in R

```
log_post <- function(theta) {          # log(likelihood × prior)
  if (theta <= 0 || theta >= 1) return(-Inf)
  dbinom(14, 20, theta, log = TRUE) + dbeta(theta, 2, 2, log = TRUE)
}

metropolis <- function(n_iter, step, start) {
  theta <- numeric(n_iter)
  current <- start
  for (i in 1:n_iter) {
    proposal  <- current + rnorm(1, mean = 0, sd = step)
    log_ratio <- log_post(proposal) - log_post(current)
    if (log(runif(1)) < log_ratio) current <- proposal   # accept
    theta[i] <- current             # record, also after a rejection
  }
  theta
}

chain <- metropolis(5000, step = 0.2, start = 0.5)
```

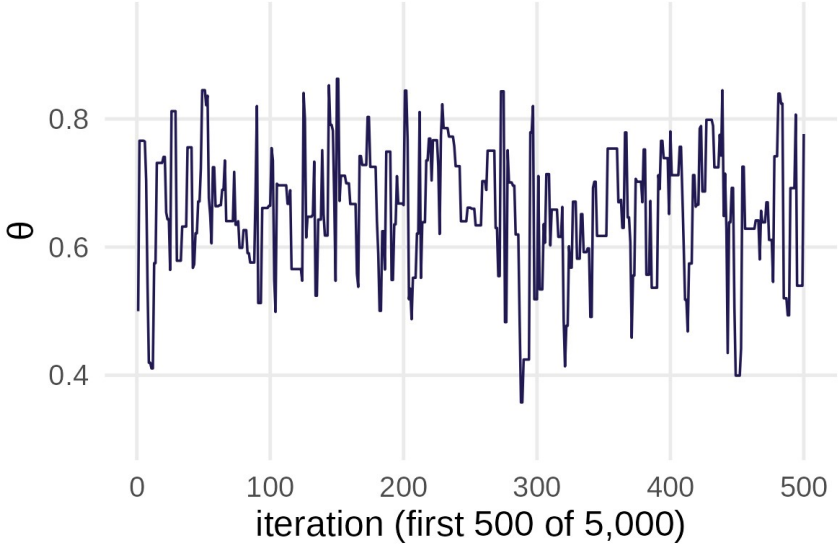
Why logs?

Heights can be tiny (1e-300). log(r) = difference of logs; compare with log(u).

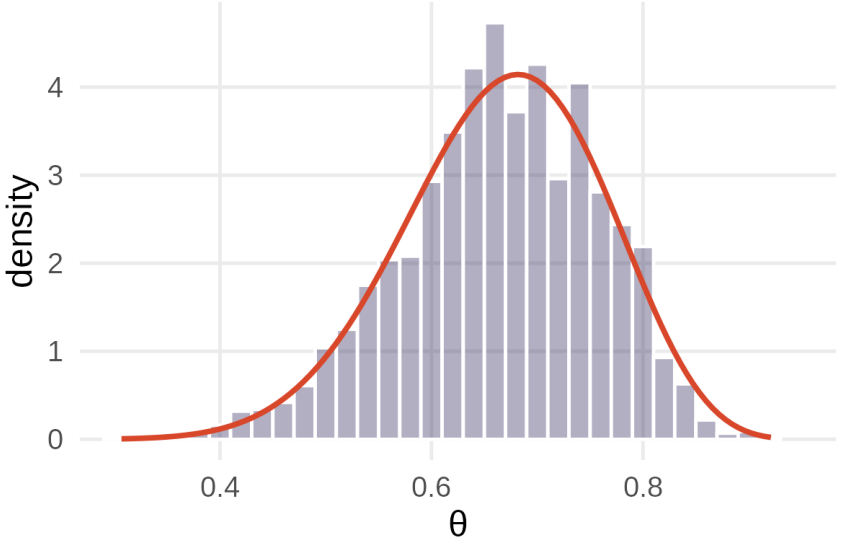
u < r

With u ~ Uniform(0, 1), “u < r” happens with probability min(1, r).

Does it work? Compare with the exact answer



Trace plot: the chain moves quickly across the whole posterior.



Histogram of 5,000 values vs Beta(16, 8) (red): mean 0.664 (exact 0.667), P(θ > 0.6) = 0.766 (exact 0.763).

Effective sample size (ESS)

Definition

ESS = the number of independent draws that would give the same Monte Carlo error as our S dependent chain values. MCSE = SD / √ESS

Autocorrelation at lag k

ρ_k = correlation between $\theta^{(t)}$ and $\theta^{(t+k)}$: how similar are draws k steps apart?

$$\mathbf{ESS = S / (1 + 2(\rho_1 + \rho_2 + \rho_3 + \ldots))}$$

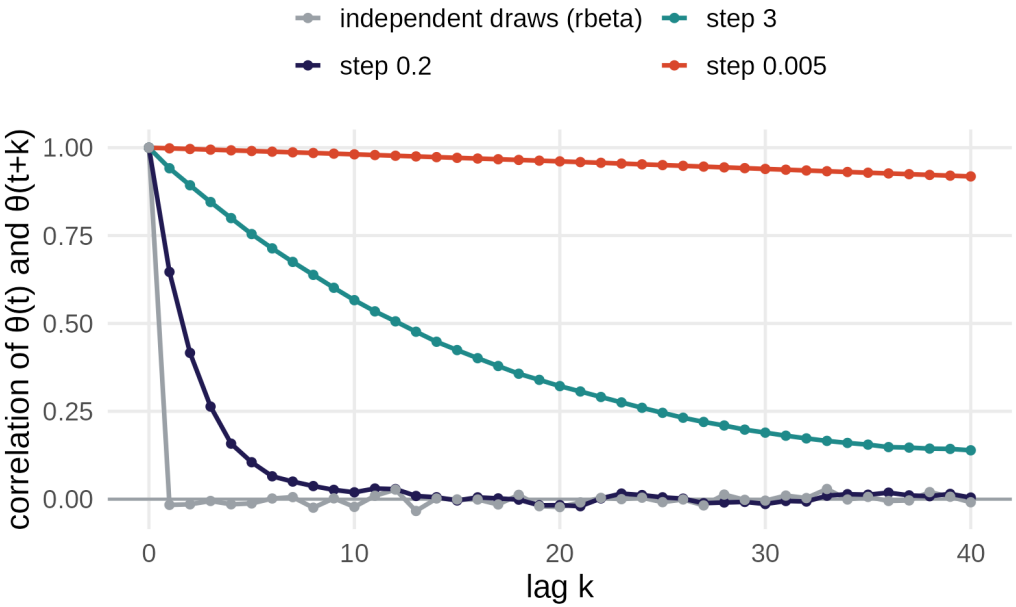
Independent draws: all $\rho_k = 0$, so ESS = S.

if $\rho_k = \rho^k$	ESS
$\rho = 0$	S
$\rho = 0.5$	S / 3
$\rho = 0.9$	S / 19

```
posterior::ess_basic(chain)
```

The more neighbouring draws resemble each other, the smaller the ESS.

Autocorrelation and ESS of our chains



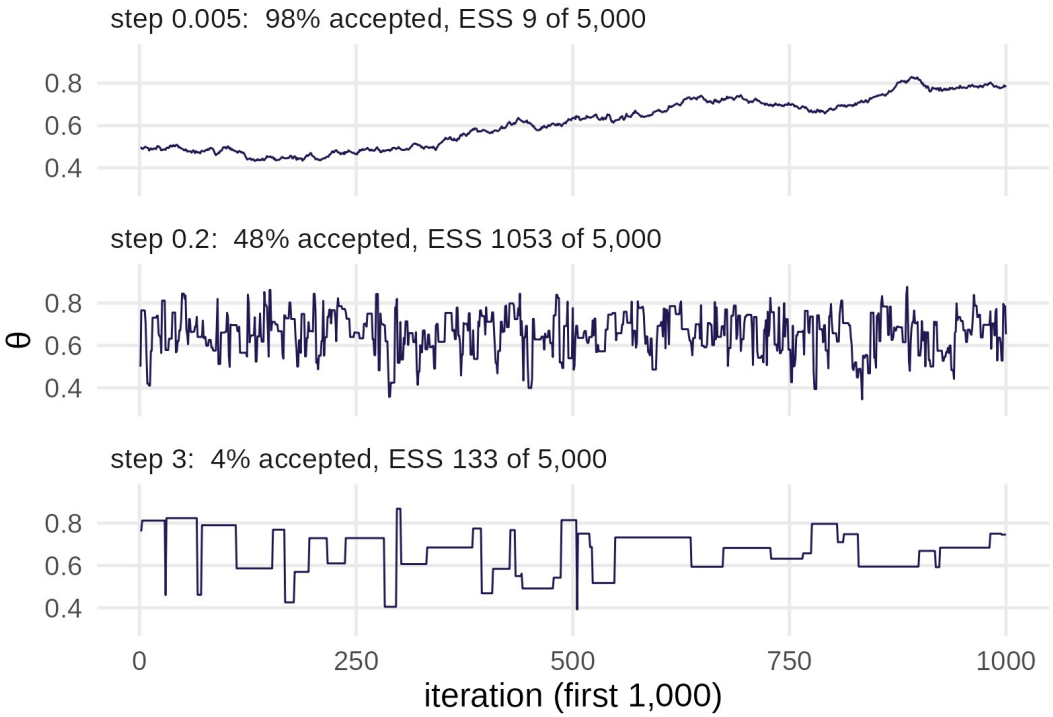
5,000 draws	ESS	MCSE of mean
independent	≈ 5,000	0.0013
step 0.2	1,053	0.0029
step 3	133	0.0090
step 0.005	9	0.031

MCSE = SD / √ESS

Slow decay of the correlation → small ESS → large MCSE. Next slide: what the three chains look like.

Neighbouring draws are similar, so 5,000 chain values carry less information than 5,000 independent ones.

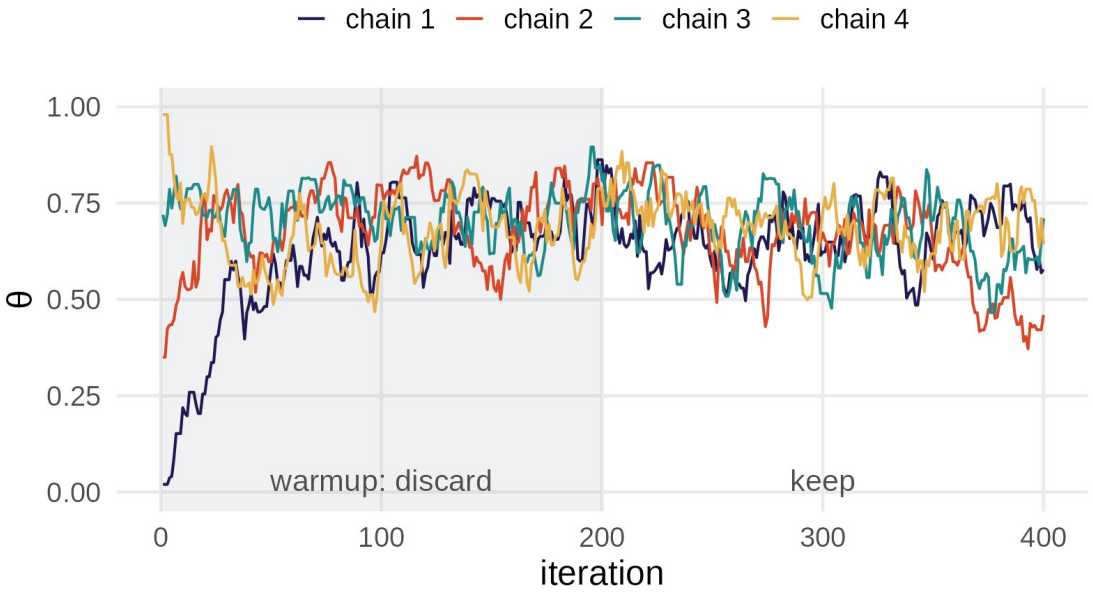
The step size matters



- Tiny steps: almost all accepted, but the chain crawls.
- Huge steps: most land in low regions and are rejected; the chain sticks.
- In between: moves far and often.

A high acceptance rate is not good news by itself.

Where to start, and when to start counting



- A bad start (θ = 0.02) needs time to reach the posterior.
- Drop the first part: warmup (burn-in).
- Run several chains from different starts.
- If they end up in the same place, that is evidence — not proof — that they found the posterior.

Worked example: how many draws?

Question 1

You run this ONCE (S = 1,000 independent draws):

```
draws <- rbeta(1000, 16, 8)
h <- draws > 0.6     # 1 or 0
mean(h)              # 0.767
sd(h)                # 0.423
```

a) Estimate the MCSE of 0.767 from this one run.
b) How many draws give MCSE 0.005?

Question 2

The step-0.2 chain (5,000 values, ESS 1,053, posterior SD 0.094) gives mean 0.664.

What is the MCSE? How does it compare with 5,000 independent draws?

2 minutes with your neighbour.

Worked example: answers

Question 1

a) One run is enough: the spread of the 0/1 values h tells us the spread of their average (slide “Where does √S come from?”).

MCSE = sd(h) / √S = 0.423 / √1000 = 0.013

b) 0.423 / √S = 0.005 → √S = 85 → S ≈ 7,200 draws

(2.7 times smaller error needs $2.7^2 \approx 7$ times more draws)

Question 2

MCSE = 0.094 / √1053 = 0.0029

With 5,000 independent draws: 0.094 / √5000 = 0.0013

The chain's error is about 2.2 times larger: it needs 5000 / 1053 ≈ 4.7 times as many iterations for the same precision.

“The estimate itself is random” repeated the experiment 1,000 times only to check the formula: SD of the estimates 0.014.

One run gives both the estimate and its MCSE. For a chain, use the ESS instead of S.

Exercise A · 09:45 – 11:10

- Monte Carlo with rbeta(): estimates, their MCSE, comparison with exact values.
- Run the supplied Metropolis sampler; find the accept step and the repeated value.
- Trace plot and histogram against the exact posterior.
- A poorly moving chain: acceptance rate, ESS and MCSE.
- Write your group's question set: concept · computation · pitfall, with worked solutions.

Everyone back at

11:10

peer review A

Take a break when it suits your group. Rotate roles: coder · checker · question writers · interviewer · reporter.

Peer review A · 11:10 – 11:55

20 min

Partner interviews

Round 1: odd group asks even group. Round 2: swap. Two questions each, to named members.

23 min

Plenary

Two groups drawn from those not yet drawn. Then the reporters' hardest open questions.

2 min

Wrap-up

What to take into lunch and Lecture B.

Partner pairs: 1 ↔ 2 · 3 ↔ 4 · 5 ↔ 6 · 7 ↔ 8 · 9 ↔ 10

```
sample(not_yet_drawn, 2)   # plenary draw
```